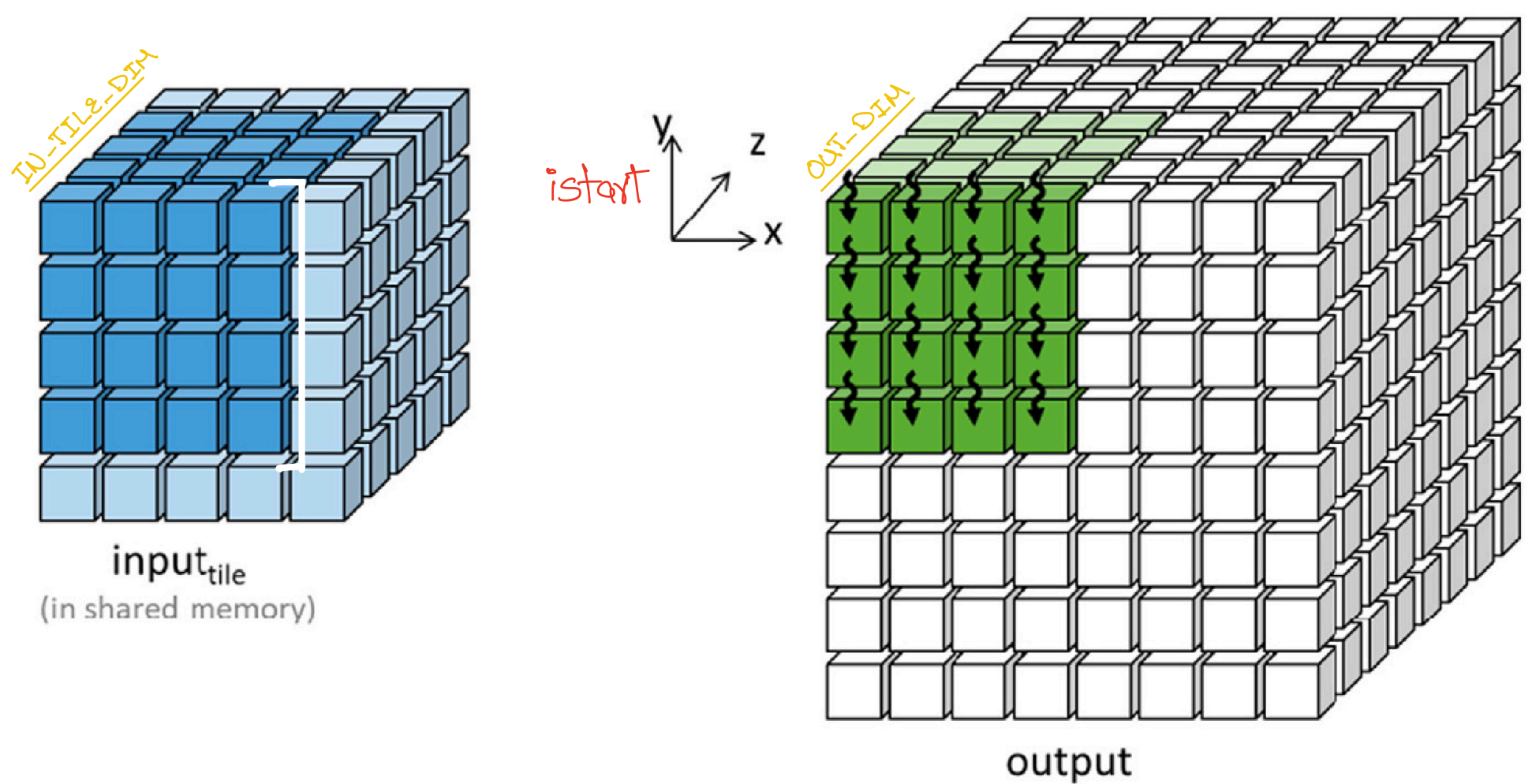
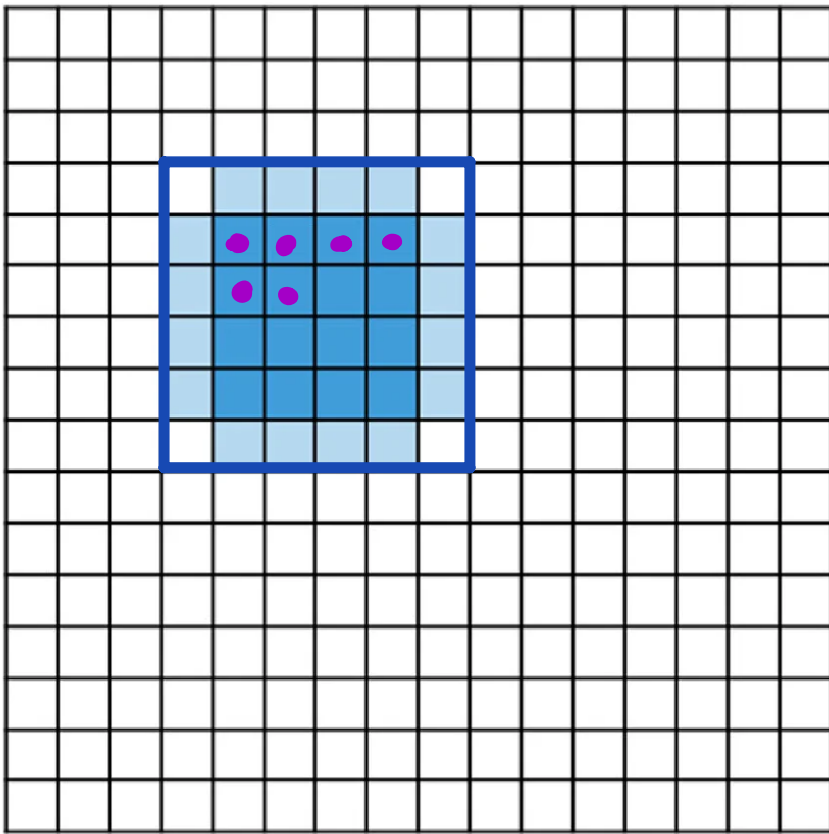


8.1 Background

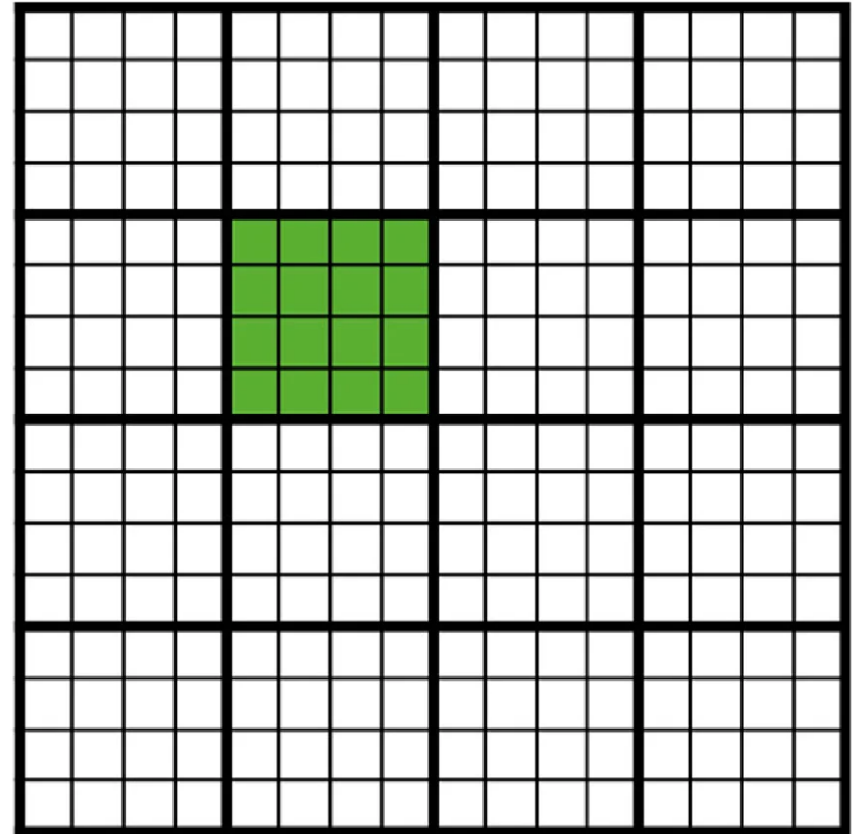
- **First derivative (f'):** Uses 1 point on each side (a 3-point stencil: $i - 1, i, i + 1$).
- **Second derivative (f''):** Uses 2 points on each side (a 5-point stencil: $i - 2, i - 1, i, i + 1, i + 2$).
- **n -th derivative ($f^{(n)}$):** Uses n points on each side.

8.4 Thread Coarsening





input
(in global memory)



output

```
__global__ void stencil_kernel_updated(float* in, float* out, unsigned int N) {
    int i = blockIdx.z*OUT_TILE_DIM + threadIdx.z - 1;
    int j = blockIdx.y*OUT_TILE_DIM + threadIdx.y - 1;
    int k = blockIdx.x*OUT_TILE_DIM + threadIdx.x - 1;

    __shared__ float in_s[IN_TILE_DIM][IN_TILE_DIM][IN_TILE_DIM];

    if (i >= 0 && i < N && j >= 0 && j < N && k >= 0 && k < N) {
        in_s[threadIdx.z][threadIdx.y][threadIdx.x] = in[i*N*N + j*N + k];
    }

    __syncthreads();

    int tileZ = threadIdx.z - 1;
    int tileY = threadIdx.y - 1;
    int tileX = threadIdx.x - 1;

    if (i >= 1 && i < N-1 && j >= 1 && j < N-1 && k >= 1 && k < N-1) {
        if (tileZ >= 0 && tileZ < OUT_TILE_DIM &&
            tileY >= 0 && tileY < OUT_TILE_DIM &&
            tileX >= 0 && tileX < OUT_TILE_DIM) {
            out[i*N*N + j*N + k] = c0*in_s[threadIdx.z][threadIdx.y][threadIdx.x]
                                   + c1*in_s[threadIdx.z][threadIdx.y][threadIdx.x-1]
                                   + c2*in_s[threadIdx.z][threadIdx.y][threadIdx.x+1]
                                   + c3*in_s[threadIdx.z][threadIdx.y-1][threadIdx.x]
                                   + c4*in_s[threadIdx.z][threadIdx.y+1][threadIdx.x]
                                   + c5*in_s[threadIdx.z-1][threadIdx.y][threadIdx.x]
                                   + c6*in_s[threadIdx.z+1][threadIdx.y][threadIdx.x];
        }
    }
}
```

} different but equivalent to what is done in the book

